

Project FLASH

An offline pneumonia screener built as a neural network in FPGA hardware

Karthikeya Reddy and Yashwant Rajesh

Code: github.com/Karthik-Flash/ProjectFlash

(hackathon version: github.com/Karthik-Flash/PneumoniaFPGA)

How numbers are labelled. We mark every figure by where it comes from, because our early material mixed these up.

Label	Meaning
Measured	Produced by a tool run on the current design and reproducible from the repository
Synthesis estimate	Reported by Vivado after synthesis, before the design is placed on the chip
Derived	Calculated from measured quantities, not yet observed on hardware
Not yet done	Planned work; no number exists

1. The project on one page

FLASH takes a chest X-ray and says whether it shows signs of pneumonia, entirely inside a low-cost AMD Zynq-7020 chip, with no internet and no GPU.

The project began at the AMD FPGA Hackathon 2026, where it placed 5th. After the hackathon we rebuilt it against an exact software reference and found five hidden bugs that had been there all along. The corrected design now reproduces the trained network bit by bit. We then redesigned it (V1) so a single set of circuits can run a larger network on full-size 224 x 224 X-rays from a real clinical dataset.

What	Current value	Source
Hardware matches trained model (v0, 28 x 28)	244 of 244 test images, exact 32-bit scores	Measured
Hardware matches trained model (V1.2, 224 x 224)	16 of 16 images exact so far; 244-image run pending	Measured
Model quality (V1.2, 4,003 unseen patients)	AUROC 0.823 (95% CI 0.808 to 0.838)	Measured
Sensitivity / specificity at chosen threshold	91.0% / 53.9%	Measured

Clock and chip usage (V1.2)	75 MHz; 3.8% of logic, 6.4% of multipliers, 57% of block RAM	Synthesis estimate
Time per image (V1.2)	About 163 ms at one multiply per clock	Derived
Running V1 on the board, power	Needs an interface wrapper	Not yet done

The next milestone is to run V1 on the programmable logic (PL) of the PYNQ-Z2 board and show the board's answers match the software reference exactly, then measure real time and power. Section 7 lays out how.

2. The problem

Pneumonia fills the small air sacs of the lungs with fluid. It is treatable, yet the WHO reports it killed 740,180 children under five in 2019, most of them in South Asia and sub-Saharan Africa. A chest X-ray is the usual way to confirm it, but someone has to read the X-ray. India has roughly one radiologist per 100,000 people, concentrated in cities, so a film taken at a small hospital can wait hours or days for a report.

Most AI tools that read X-rays run on a server or in the cloud. That depends on a reliable connection and sends patient images off site. Our aim is a small, low-power device that reads the film where it is taken and gives an immediate first opinion: "likely normal" or "send this for an urgent read". It is meant to help a clinician prioritise, not to replace a radiologist.

We chose an FPGA rather than a phone or a small computer for one main reason: the chip computes exactly what we validated, every time, with no operating system or floating-point library in the way. That property is easy to test and easy to explain to a regulator. Low power is the second reason, and it still has to be measured for V1.

3. Background, from scratch

3.1 An X-ray is a grid of numbers

An X-ray image is stored as a grid of pixels, each a brightness value. Air in healthy lungs looks dark; fluid, as in pneumonia, looks like a hazy bright patch called an opacity. Hospital scanners save images as DICOM files with 10 to 16 bits per pixel. Before our network sees an image, the software converts it to 8 bits per pixel (0 is black, 255 is white) and resizes it to 224 x 224, which is 50,176 numbers.

3.2 What a neural network does

A neural network is a function with many adjustable numbers called weights. We show it thousands of X-rays with known answers and nudge the weights until its answers improve; this

is training. The data is split into three parts that must never mix: a training set to adjust weights, a validation set to make choices such as when to stop, and a test set used once at the end to report results. For medical images the split is by patient, so no person appears in two parts.

Each artificial neuron multiplies its inputs by weights, adds them up with a constant called a bias, and passes the result through an activation. We use ReLU, which turns negative values into zero. The last layer gives two scores called logits, one for "normal" and one for "pneumonia". Our decision uses the difference between them, the margin.

3.3 Convolution, the core operation

A convolutional neural network (CNN) slides a small 3 x 3 grid of weights, called a filter, across the image. At each position it multiplies the nine pixels underneath by the nine weights and adds them. Here is one position, using a filter that responds to a left-to-right brightening:

Pixels under the filter	Filter weights	Products
10 20 30	-1 0 +1	-10 0 +30
40 50 60	-1 0 +1	-40 0 +60
70 80 90	-1 0 +1	-70 0 +90

(This is an example)

The products add up to 60; with a bias of 5 the output is 65. Repeating this at every position produces a new grid called a feature map. A layer with 8 filters produces 8 feature maps (8 channels). Moving the filter two pixels at a time (stride 2) halves the width and height. Early layers pick up edges, later layers combine them into larger patterns such as a dense region in one lung. One multiply plus one add is called a multiply-accumulate (MAC); our network needs about 12.2 million of them per image.

3.4 From decimals to 8-bit integers

Networks are normally trained with 32-bit decimal numbers. Decimal arithmetic is expensive in hardware, so we use small integers: weights between -127 and 127 and pixel-like values between 0 and 255. This is called quantization. Rounding a finished network usually costs accuracy, so we train the network directly in integer units from the start. The numbers the chip stores are the trained weights themselves, which means there is nothing to convert and nothing to get wrong in between. Between layers the sums grow large, so each layer divides by a power of two (a bit shift, which costs nothing in hardware) and clamps the result back into 0 to 255.

3.5 What an FPGA is, and the two halves of a Zynq

A normal processor runs one instruction after another on fixed circuits. An FPGA is a chip of unconnected logic that you wire up yourself, so you can build exactly the circuit a task needs and run all of it in parallel on every clock tick. Its main building blocks are look-up tables (LUTs, small logic gates), flip-flops (1-bit memory cells), DSP blocks (hard multipliers) and block RAM

(on-chip memory, about 630 KB on our chip). Our design runs at 75 MHz, so one clock tick is 13.3 nanoseconds.

The Zynq-7020 on the PYNQ-Z2 board has two halves. The **PS** (processing system) is a pair of ARM cores running Linux, plus the controller for the board's DDR memory. The **PL** (programmable logic) is the FPGA fabric where our accelerator lives. They talk over AXI, ARM's standard on-chip bus: AXI-Lite for reading and writing small registers ("start", "threshold", "result"), AXI-Stream for a flow of data such as pixels, and an AXI-DMA block that copies a buffer from DDR into a stream without the ARM touching each byte.

3.6 How a hardware design becomes a working chip

1. **RTL design:** describe the circuit in Verilog, saying what is stored in registers and how values move each clock tick.
2. **Simulation:** run the Verilog in software with a testbench that feeds inputs and checks outputs. This is where we prove the arithmetic is exact.
3. **Synthesis:** Vivado turns the Verilog into LUTs, flip-flops, DSPs and RAMs and estimates size and speed.
4. **Implementation:** Vivado places every element on the chip and wires it up. Only now are timing and size final.
5. **Bitstream:** the configuration file that is loaded onto the board.

V1 has completed steps 1 to 3. Step 4 needs the interface wrapper described in Section 7.

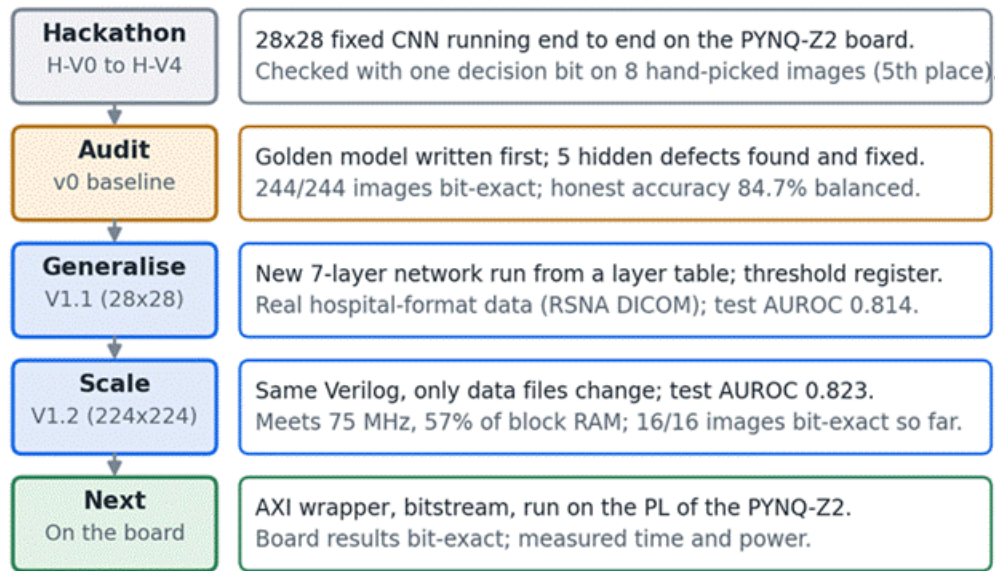
3.7 How we measure accuracy

Measure	Plain meaning
Sensitivity	Of the patients who have pneumonia, the share we flag
Specificity	Of the patients who do not, the share we clear
PPV / NPV	Of those flagged, the share truly sick / of those cleared, the share truly healthy
AUROC	Chance that a random sick patient scores higher than a random healthy one (0.5 is a coin toss, 1.0 is perfect). It does not depend on where the threshold is set.

4. How the project got here

The design went through nine versions in three phases: five during the hackathon that made it work, an audit that made it correct, and the V1 stages that made it general. We write the hackathon versions as H-V0 to H-V4 to keep them apart from the later v0 and V1.

How the design got here: make it work, make it correct, make it general



The hackathon's 95.4% accuracy came from a datapath with bugs and is no longer quoted.

Figure 1. The path from the hackathon prototype to V1 and the next step on hardware.

4.1 During the hackathon

Version	What changed	What we learned
H-V0	Feature maps kept in 62,720 flip-flops; the design could not be placed on the chip. Moving them to block RAM fixed it.	Large arrays belong in block RAM
H-V1	Values overflowed silently between stages; we widened buses and added a shift after pooling.	Integer overflow gives wrong answers without any warning
H-V2	Stopped the multiplier pipeline from switching when idle. Vivado's power estimate fell from 35.9 W to 7.2 W.	Both figures were default estimates, not measurements
H-V3	Simulation passed 5 of 8 test images; we blamed the failures on the model.	They were in fact a hardware bug (see 4.2)
H-V4	Accelerator connected to the ARM over AXI and run on the board: image in, decision out.	The full system worked end to end

4.2 The audit: five hidden bugs

After the hackathon we wrote a golden model first: a small NumPy program that computes exactly what the hardware should compute, integer for integer. We then dumped every intermediate result out of the simulator and compared it with the golden model. Five defects appeared.

Where	Bug	Effect
Line buffer	One of the nine window pixels was read before it had been written	Every 3 x 3 window had one wrong pixel
Line buffer	Windows were centred one row too early	The last image row was never used; the whole feature map shifted
Max-pool	Memory data captured one clock too early	Pooling compared the wrong values
Dense layer	Read pointer off by one	One input skipped, another used twice
Testbench	Inputs driven on the same clock edge the design samples; result read one clock early	Tests read the previous image's answer

The reason none of this was caught: the old test checked a single yes/no bit on eight images, and those eight had been chosen because the model was very confident about them. A confident model still gives the right yes/no even when the arithmetic underneath is wrong. The new test compares the full 32-bit scores on 244 images chosen without looking at confidence, including 46 the model gets wrong. The hardware has to reproduce the model's mistakes too, because a matching 32-bit score cannot happen by accident.

4.3 The v0 baseline

With the bugs fixed, the 28 x 28 design matches the golden model on 244 of 244 images, and every intermediate layer matches too. After placement on the chip it meets 75 MHz with room to spare (about 84 MHz possible), and a power analysis using real switching activity from simulation gives 0.116 W for the chip. The honest accuracy is 84.7% balanced accuracy on PneumoniaMNIST, lower than the 95.4% shown at the hackathon, which the buggy hardware could never have produced.

4.4 Why we redesigned it (V1)

- In v0, 99% of the weights sat in one dense layer whose size was tied to the 28 x 28 image, so it could not grow to clinical resolution.
- Each layer was a separate hand-written module, so any change to the network meant new Verilog.
- The decision was a plain comparison of two scores with no way to adjust it after training.
- PneumoniaMNIST is tiny and pre-shrunk; we needed real hospital-format images at full resolution.

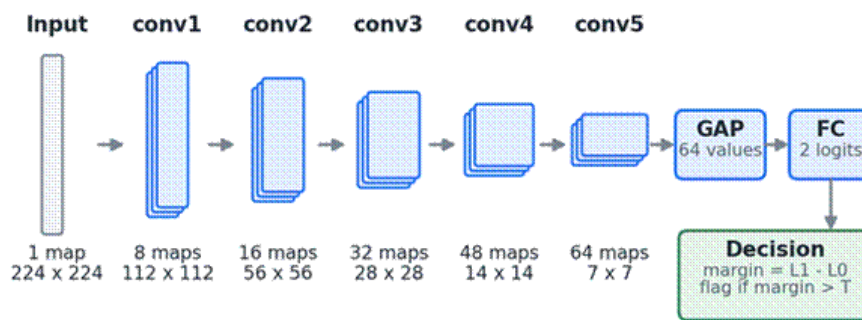
5. The current CNN pipeline (V1)

5.1 Data and preprocessing

V1 trains on the RSNA Pneumonia Detection dataset: 26,684 adult patients, one frontal X-ray each, labelled by radiologists and stored as DICOM files. We split it by patient into 18,678 for training, 4,003 for validation and 4,003 for testing; 22.5% of patients are positive. Preprocessing follows the DICOM standard's own steps (rescale, window, invert images stored as white-is-zero, letterbox so the chest is not stretched, resize). We tested it on genuine scanner files, not only on RSNA's already-simplified images, and the same function runs on the board's ARM so the hardware sees exactly the pixel training saw.

5.2 The network

The V1 network: the picture shrinks while the number of feature maps grows



Each conv layer: 3x3 filter, stride 2, padding 1, then shift right and clamp to 0..255 (acts as ReLU).
Total: 47,602 INT8 parameters, all stored on-chip, and 12.19 million multiply-accumulates per image.

Figure 2. The V1 network at 224 x 224.

Layer	Operation	Output at 224 x 224	Parameters	MACs per image
conv1	3 x 3, stride 2, 1 to 8 channels	8 x 112 x 112	80	903,168
conv2	3 x 3, stride 2, 8 to 16	16 x 56 x 56	1,168	3,612,672
conv3	3 x 3, stride 2, 16 to 32	32 x 28 x 28	4,640	3,612,672
conv4	3 x 3, stride 2, 32 to 48	48 x 14 x 14	13,872	2,709,504
conv5	3 x 3, stride 2, 48 to 64	64 x 7 x 7	27,712	1,354,752
GAP	average of each channel	64	0	small
FC	dense, 64 to 2	2 logits	130	128
Total			47,602	12,192,896

Three design choices matter. Stride-2 convolutions shrink the image as they go, which does a quarter of the work of a normal convolution and removes the separate pooling step where one of the v0 bugs lived. Global average pooling (GAP) reduces each of the 64 final feature maps to one number, so the classifier always sees 64 values whatever the image size; the same hardware therefore runs 28 x 28 and 224 x 224. And 47,602 eight-bit weights take about 47 KB, small enough to keep permanently on the chip.

5.3 The integer rules shared by training, reference and hardware

- Pixels and activations are unsigned 8-bit (0 to 255); weights are signed 8-bit (-127 to 127); biases are 32-bit.
- Each convolution output = bias + sum of (weight x pixel) over the 3 x 3 window and all input channels, held in a 22-bit accumulator.
- The sum is shifted right by a per-layer amount (7 or 8) and clamped to 0..255. The clamp also acts as ReLU.
- GAP adds up each 7 x 7 map and shifts right by 6 (divide by 64 instead of 49, which is a wire instead of a divider; training sees the same thing).
- The two logits come from a 64-input dense layer; margin = logit(pneumonia) minus logit(normal); the image is flagged if margin is greater than T.

5.4 Training and the decision threshold

Training is how the network learns. We show it thousands of labelled X-rays and adjust its weights a little after each pass. One full pass through all the training images is called an **epoch**.

We train in two phases:

1. **Normal training.** The weights can take any decimal value.
2. **Integer training.** The weights are rounded to whole numbers on every pass. This lets the network get used to the integer arithmetic the chip uses, before we put it on the chip.

After each epoch we test the network on the **validation set**. This is a separate group of patients used only for making choices, never for learning. We keep the version that scores best there.

The network does not give a yes or no. It gives a score. A high score means "this looks like pneumonia". The **threshold T** is the cut-off: scores above T are flagged. We chose T on the validation set so that the network catches at least 90 out of every 100 pneumonia cases. Then we locked T before looking at the final test patients, so the results below are a fair test.

T is stored in a register on the chip, a small memory slot the software can write to. A clinic could later move T up or down without retraining. Raising T means fewer false alarms but more missed cases. Lowering it does the opposite.

5.5 Model results

These results come from 4,003 patients the network never saw during training. They are computed by our **golden model**, a reference program that does exactly the same integer arithmetic as the chip. So these are the numbers the hardware will produce, not a separate software estimate.

First, what each measure means:

- **AUROC**: how well the network ranks sick patients above healthy ones. 0.5 is a coin toss and 1.0 is perfect. It does not depend on where T is set.
- **Confidence interval**: the range the true AUROC most likely falls in, given how many patients we tested.
- **Sensitivity**: of the patients who have pneumonia, the share we flag.
- **Specificity**: of the patients who do not, the share we correctly clear.
- **PPV** (positive predictive value): of the patients we flag, the share who really have pneumonia.
- **NPV** (negative predictive value): of the patients we clear, the share who are really healthy.

Test set: 4,003 patients	Small images (28 x 28)	Full-size images (224 x 224)
AUROC (likely range)	0.814 (0.799 to 0.830)	0.823 (0.808 to 0.838)
Sensitivity / specificity at T	89.1% / 54.7%	91.0% / 53.9%
PPV / NPV at T	36.4% / 94.5%	36.5% / 95.4%
AUROC: pneumonia vs clearly normal lungs	0.933	0.936
AUROC: pneumonia vs other lung problems	0.726	0.738
AUROC using only the type of X-ray film	0.706	0.706

What these results tell us:

- **Bigger images help a little.** The full-size version is better on almost every measure, but the AUROC gain is small and the likely ranges overlap.
- **A "clear" result can be trusted; a "flag" means "get this checked".** About 95% of the patients we clear are truly healthy. But to catch 91% of pneumonia cases, we also flag about 46% of healthy patients.
- **Easy cases go well, hard cases less so.** The network separates pneumonia from clearly normal lungs well (0.936). It struggles more to separate pneumonia from other lung problems, such as fluid or nodules (0.738).
- **Part of the score comes from a shortcut.** Chest X-rays are taken in two ways:
 - **AP** (front to back) films are usually taken at the bedside of sicker, admitted patients.
 - **PA** (back to front) films are usually taken of patients standing up who walk in.
- Just knowing which type of film it is already scores 0.706, because sicker patients get more AP films. When we test on only one film type at a time, the network scores 0.74 to 0.77. So part of the 0.823 comes from the film type rather than the lungs themselves.
- **No meaningful difference between male and female patients.**

6. The current hardware (V1)

V1 replaces the fixed v0 circuit with a small engine: one convolution unit that is reused for all five convolution layers, driven by a table in on-chip memory that describes each layer. Figure 3 follows one X-ray through the full system as it will run once connected to the board.

One X-ray through the system: the ARM prepares, the FPGA fabric computes

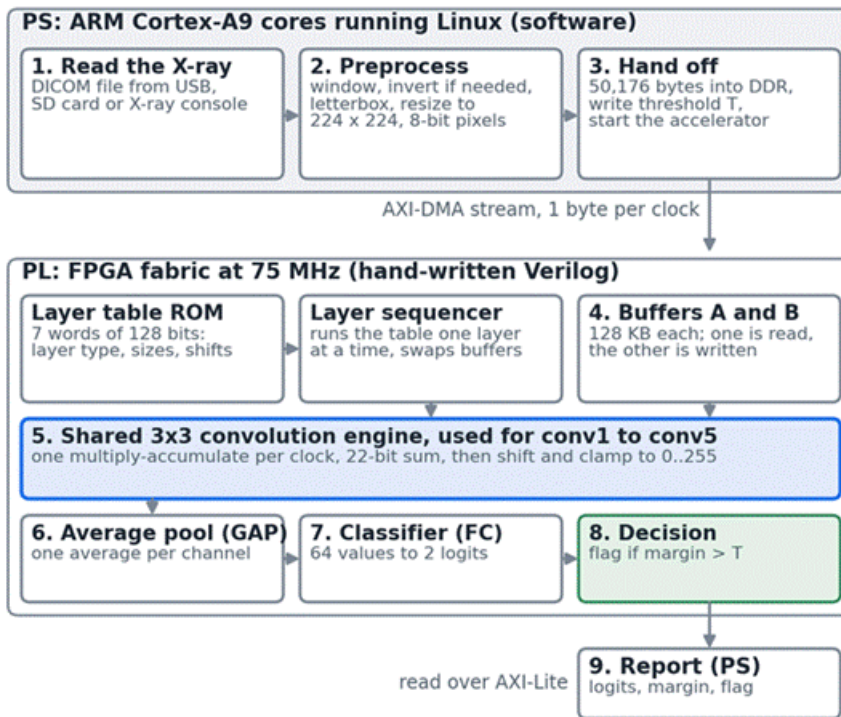


Figure 3. One X-ray through the system. Steps 1 to 3 and 9 are software on the ARM; steps 4 to 8 are our Verilog on the FPGA fabric.

6.1 The modules

Module	What it does
layer_seq	Reads the layer table one entry at a time, starts the right unit, and swaps the two feature-map buffers between layers
conv_engine	The shared 3 x 3 stride-2 convolution. One multiply-accumulate per clock, pipelined in stages, 22-bit sum, then shift and clamp
fmap_ram (x2)	Buffers A and B, 128 KB each. Each layer reads one and writes the other ("ping-pong")
gap_unit	64 running sums over the last feature map, then shift
fc_unit	The 64 to 2 dense layer, producing two 32-bit logits
decision	Computes the margin and compares it with the threshold register
top_v1	Connects everything, holds the weight, bias and layer-table memories, and lines up memory read delays
line_buffer_v1	A streaming window generator, built and tested but not yet used; ready for a faster engine later

6.2 The layer table

Each layer is one 128-bit word holding its type (convolution, pooling or dense), filter size, stride, padding, channel counts, image sizes, shift amount and where its weights start in memory. To change the network, we change this table and the weight file; the Verilog stays the same. That is how the same circuit runs both 28 x 28 and 224 x 224.

6.3 Size and speed

Synthesis estimate at 75 MHz	V1.1 (28 x 28)	V1.2 (224 x 224)	Available
Timing margin on the slowest path	+0.093 ns	+0.085 ns	must be positive
Logic (LUTs)	1,907	2,006	53,200
Flip-flops	2,323	2,840	106,400
DSP multipliers	14	14	220
Block RAM (36 Kb tiles)	48	80	140

The chip is mostly empty except for memory. Block RAM is the limit, mainly because of the two 128 KB buffers. The slowest path is address arithmetic (working out where a pixel sits in memory), not the multiply itself. Only 14 of 220 multipliers are used, which leaves a lot of room to run several channels in parallel later. At one multiply per clock, 12.19 million MACs take about 163 ms per image (derived).

6.4 How we know it is correct

1. **Trained model = golden model.** Because training is done in integer units, the exported weights are the model. An independent NumPy program that reads only the exported files gives identical scores on all 4,003 test patients.
2. **Golden model = hardware.** The simulator dumps every layer's output and every raw sum; Python compares them with the golden model. A mismatch points to the exact layer. V1.2 so far: 16 of 16 images exact, including all intermediate traces.
3. **Reported accuracy = hardware accuracy.** Given 1 and 2, the results in Section 5.5 are what the chip will output.

We also check that no sum can overflow for any possible input image, using the actual trained weights, and we record the ranges seen on real data. Every file the hardware reads is generated by one notebook, so nothing is typed by hand.

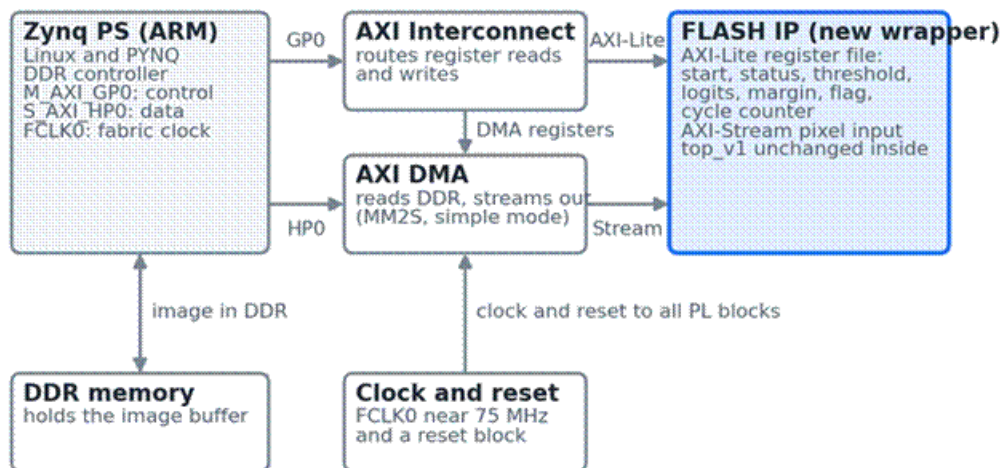
6.5 What is not done yet

- The full 244-image hardware comparison at 224 x 224 (16 images so far).
- The interface wrapper, placement on the chip, bitstream and a run on the board.
- Measured time per image and measured power for V1.
- Testing on a second dataset from a different hospital, and on children.

7. Next steps: running V1 on the board

The goal of this stage is simple to state: the PYNQ-Z2 board, given the same 244 test images, returns exactly the same 32-bit scores as the golden model. Once that holds, we measure how long one image takes and how much power the board uses. Only one new piece of Verilog is needed, a wrapper around the existing top_v1; everything else is standard Vivado IP.

Next step: the Zynq block design that puts FLASH on the board (proposed)



Only the wrapper is new Verilog. The DMA, interconnect and reset blocks are standard Vivado IP.

Figure 4. Proposed block design for running FLASH on the PL of the PYNQ-Z2.

7.1 Plan

1. **Finish simulation first.** Run the full 244-image comparison at 224 x 224 and fix the file-hash gap in the export notebook, so the board has a trusted reference to match.
2. **Write the wrapper.** An AXI-Stream input that feeds pixels to top_v1 one per clock, and an AXI-Lite register file for control and results (proposed map below). Add a cycle counter that runs from start to done, which gives us exact hardware latency for free.
3. **Simulate the wrapper** with AXI verification IP or a simple testbench before going near the board.
4. **Package it as IP and build the block design:** Zynq PS, AXI DMA (read channel only), AXI interconnect, reset block and our IP, as in Figure 4.
5. **Implement and check timing** after placement and routing, not just synthesis. Record final LUT, flip-flop, DSP and block RAM usage.
6. **Generate the bitstream** and load it through PYNQ as an overlay.
7. **Write the driver in Python:** preprocess the DICOM, copy pixels into a DMA buffer, write T, start, send the buffer, wait for done, read logits, margin, flag and cycle count.

8. **Prove it on the board:** run all 244 verification images and compare every score with the expected files. Then run the full 4,003-patient test set on the board, so the published accuracy is literally what the board produced.
9. **Measure:** latency from the cycle counter and from wall-clock time including the DMA; board power with a USB power meter, idle and while running; chip power from Vivado using switching activity recorded in a post-implementation simulation.
10. **Compare fairly:** run the same integer model on the board's own ARM cores and on a Raspberry Pi-class computer, so we can say honestly what the FPGA adds.

Glossary

Term	Meaning
Accumulator	Register holding a running sum; it must be wide enough never to overflow
AP / PA film	Bedside front-to-back X-ray / standing back-to-front X-ray
AXI, AXI-Lite, AXI-Stream, DMA	On-chip bus family: registers, data streams, and a memory-to-stream copier
Bit-exact	Two computations produce identical integers
Block RAM	Dedicated memory blocks inside the FPGA
DICOM	Standard file format for medical images
DSP block	Hard multiplier-adder inside the FPGA
GAP	Global average pooling: one average per feature map
Golden model	Independent reference program the hardware is checked against
Logit / margin	Raw class score / difference between the two scores
LUT	Look-up table, the basic logic cell of an FPGA
MAC	One multiply and one add
PS / PL	ARM processing system / programmable logic (FPGA fabric) of a Zynq
ReLU	Activation that turns negatives into zero
RTL / Verilog	Hardware description at the level of registers, and the language we write it in
Stride	How far the filter moves between positions